



Bil204 - Yazılım Mühendisliği

Proje Tasarım Raporu

Yazılımci Birleştirme Platformu

(DevSozluk)

Grup Üyeleri:

Tevfik Gorel - 25100011407

Mert Sefa Tas - 25100011416

Sefa Koyuncu - 24100011023

Hamza Erguven - 23100011071

Bilgisayar Mühendisliği Bölümü

Necmettin Erbakan Üniversitesi

Teslim Tarihi: 04.05.2026

İçindekiler

1. Giriş	3
1.1. Sistemin Amacı	3
1.2. Tasarım Hedefleri	4
1.2.1. Yüksek Performans Ve Dinamik Ölçeklenebilirlik	4
1.2.2. Güvenlik Ve Veri İzolasyonu	4
1.2.3. Bakım Yapılabilirlik Ve Esneklik	5
1.2.4. Kullanılabilirlik ve Bilişsel Yükün Azaltılması	5
2. Üst Düzey Yazılım Mimarisi	5
2.1. Alt Sistem Ayrıştırması	6
2.2. Donanım/Yazılım Eşlemesi	8
2.3. Kalıcı Veri Yönetimi	9
2.4. Erişim Kontrolü ve Güvenlik	10
2.5. Küresel Kontrol Akışı	11
2.6. Sınır Koşulları	12
2.6.1. Başlatma	13
2.6.2. Sonlandırma	13
2.6.3. Hata Durumları	13
3. Alt Düzey Tasarım	14
3.1. Nesne Tasarım Kararları	14
3.2. Son Nesne Tasarımı	15
3.3. Paketler / Katmanlar	16
3.3.1. Dış Paketler	16
3.3.2. İç Paketler	16
3.4. Sınıf Arayüzleri	17
3.5. Tasarım Desenleri	18
Sözlük (Glossary)	19
Referanslar	20

1. Giriş

Modern yazılım mühendisliği disiplini, yalnızca kod yazma eyleminden ibaret olmayıp; karmaşık sistemlerin tasarımı, sürdürülebilir mimarilerin inşası ve karşılaşılan algoritmik problemlerin kolektif bir bilgi birikimi ile çözülmesi süreçlerini kapsayan çok boyutlu bir alandır. Günümüzde yazılım projelerinin ölçeği ve kullanılan teknolojilerin çeşitliliği hızla artarken, geliştiricilerin karşılaştıkları teknik zorluklar da aynı oranda karmaşılaşmaktadır. Bu bağlamda, bilgiye hızlı ve doğru bir şekilde erişmek, yazılım yaşam döngüsünün (Software Development Life Cycle - SDLC) en kritik aşamalarından biri haline gelmiştir. Elinizdeki bu Proje Tasarım Raporu, yazılım geliştiricilerin bilgi arayışı süreçlerindeki verimsizlikleri ortadan kaldırmayı hedefleyen "Yazılımcı Birleştirme Platformu" (DevSözlük) projesinin kavramsal analizden somut sistem mimarisine geçişini detaylandırmak amacıyla hazırlanmıştır. Bu bölümde, projenin ortaya çıkış motivasyonu, endüstride çözmeyi vaat ettiği temel problemler ve sistemin inşasında kılavuzluk edecek olan üst düzey kalite hedefleri (işlevsel olmayan gereksinimler) kapsamlı bir şekilde ele alınmaktadır.

1.1. Sistemin Amacı

Yazılım ekosisteminde teknoloji yığınlarının (tech stack) sürekli güncellenmesi, yeni framework'lerin ortaya çıkması ve mikroservisler, yapay zeka entegrasyonları gibi ileri düzey mimarilerin standart hale gelmesi, geliştiricileri sürekli bir öğrenme ve problem çözme döngüsüne itmektedir. Ancak, bir geliştirici spesifik bir sistem hatası (bug), bir arka plan (backend) darboğazı veya mimari bir ikilem ile karşılaştığında başvurduğu mevcut platformlar, genellikle beklenen verimliliği sağlayamamaktadır. Geleneksel forum sitelerinin moderasyon eksiklikleri, genel soru-cevap platformlarının dağınık yapısı ve sosyal medyanın yüzeysel iletişim biçimi, teknik bilginin parçalanmasına ve spesifik bağlamından kopmasına neden olmaktadır. Geliştiriciler, konuyla ilgisiz yüzlerce mesaj veya güncelliğini yitirmiş eski kod blokları arasında kaybolmakta; bu durum, çevik (agile) proje yönetimi süreçlerinde öngörülemeyen zaman kayıplarına ve maliyet artışlarına yol açmaktadır.

DevSözlük (Yazılımcı Birleştirme Platformu) projesinin temel amacı; tüm bu bilgi kirliliğini ve organizasyon eksikliğini ortadan kaldırarak, yalnızca yazılım geliştiricilere özel, odaklı ve etkileşimli bir "sözlük-forum" hibrit platformu yaratmaktır. Bu sistem, sıradan bir tartışma panosunun ötesine geçerek; kullanıcıların spesifik konu başlıkları (topic) altında teknik problemlerini detaylandırmalarına, mimari tartışmalar başlatmalarına ve bu bağlamda yapılandırılmış düz metinler ile kod parçacıkları (code

snippets) paylaşımlarına olanak tanımaktadır. Platformun sunduğu oylama mekanizması sayesinde, konuya hakim tecrübeli (senior) uzmanların ürettiği en doğru, en güvenli ve en optimize çözümler topluluk süzgecinden geçerek öne çıkacak ve bilgi yığınları arasında kaybolmaktan kurtulacaktır.

Nihai olarak sistem, anlık bir soru-cevap platformu olmanın çok ötesinde, yazılımcılar arasında güçlü bir profesyonel ağ ve mesleki dayanışma kültürü oluşturmayı vizyon edinmiştir. Geliştirilen bu yapı, çözülmüş ve topluluk tarafından doğrulanmış teknik problemleri indeksleyerek, gelecekte aynı sorunla karşılaşacak olan mühendisler için yüksek performanslı, aranabilir ve kalıcı bir bilgi kütüphanesi (arşiv) işlevi görecektir.

1.2. Tasarım Hedefleri

Bir yazılım sisteminin başarısı, yalnızca işlevsel gereksinimleri (functional requirements) yerine getirmesiyle değil, aynı zamanda sistemin mimari kalitesini belirleyen işlevsel olmayan gereksinimleri (non-functional requirements) ne kadar iyi karşıladığıyla doğrudan ilişkilidir. DevSözlük platformunun sürdürülebilir, güvenilir ve yüksek performanslı bir bilgi kütüphanesi olarak yıllarca hizmet verebilmesi için ISO/IEC 25010 Yazılım Kalite Modelleri referans alınarak aşağıdaki kritik tasarım hedefleri belirlenmiştir.

1.2.1. Yüksek Performans ve Dinamik Ölçeklenebilirlik (Performance & Scalability)

Sistemin hedef kitlesinin binlerce yazılımcı olduğu ve platformun doğası gereği yazma işleminden (yeni entry girilmesi) çok daha fazla okuma işlemi (başlıkların aranması ve okunması) alacağı öngörülmektedir. Bu "read-heavy" (okuma yoğunluklu) veri modeli, performansın en kritik tasarım hedefi olmasını zorunlu kılmaktadır. Sistem, ana sayfa yüklenme sürelerini ve arama motoru sorgu yanıtlarını (latency) milisaniyeler seviyesinde tutacak şekilde tasarlanmalıdır. Bunu sağlamak adına veritabanı şemaları gereksiz ilişkisel sorgulardan arındırılacak, sayfalama (pagination) algoritmaları entegre edilerek istemciye binen veri yükü minimize edilecektir. Olası trafik sıçramalarında (spike) sistemin çökmemesi için yatay ölçeklenmeye (horizontal scaling) uygun, durumsuz (stateless) bir arka plan mimarisi kurgulanacaktır.

1.2.2. Güvenlik ve Veri İzolasyonu (Security & Data Integrity)

Platformun hedef kitlesinin yazılım mühendisleri ve siber güvenlik uzmanları dahi olabileceği göz önüne alındığında, sistemin mimari güvenliği sıradan bir web uygulamasından çok daha katı standartlara sahip olmalıdır. Veritabanı güvenliği, tasarımın merkezindedir. Tüm kullanıcı şifreleri, sistemde kesinlikle düz metin (plain-text) halinde barındırılmayacak; veriler, brute-force ve rainbow table saldırılarına karşı dirençli modern, tek yönlü özetleme (hash) algoritmaları kullanılarak saklanacaktır. Ayrıca platform, kullanıcıların doğrudan kod parçacığı paylaşmasına izin vereceği için, XSS (Cross-Site Scripting) ve SQL Injection gibi zafiyetleri önlemek adına tüm kullanıcı girdileri sunucu tarafında (backend) çok katmanlı sanitizasyon (temizleme) süreçlerinden geçirilecektir.

1.2.3. Bakım Yapılabilirlik ve Esneklik (Maintainability & Extensibility)

Uzun vadeli bir proje olan DevSözlük'ün kod tabanı, gelecekteki teknolojik entegrasyonlara açık olmalıdır. Sistem; sunum, iş mantığı ve veri erişim katmanlarının birbirinden tamamen izole edildiği bir mimari ile tasarlanacaktır. Bu düşük bağımlılıklı (low coupling) yapı, arka planda yapılacak büyük mimari değişikliklerin kullanıcı arayüzünü etkilemesini engelleyecektir. En önemli genişletilebilirlik hedeflerinden biri de sistemin yapay zeka (AI) entegrasyonuna hazır olmasıdır. İlerleyen fazlarda platforma eklenebilecek; benzer içerikleri birleştiren doğal dil işleme (NLP) algoritmaları, otomatik kod analizi veya yapay zeka destekli akıllı arama modülleri, mevcut iş mantığı katmanını bozmadan sisteme mikroservisler halinde kolayca eklenebilecek bir esneklikte dizayn edilecektir.

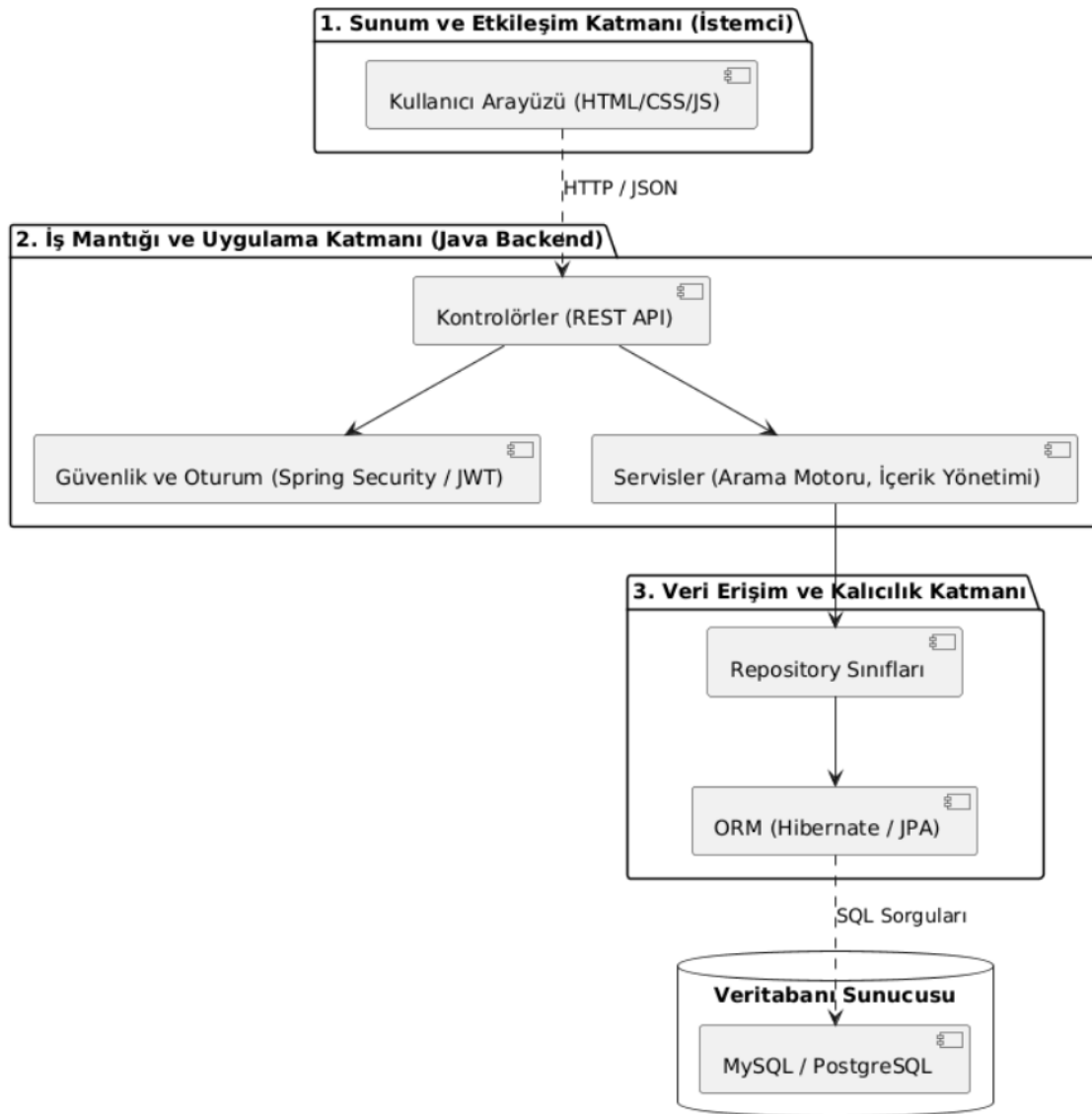
1.2.4. Kullanılabilirlik ve Bilişsel Yükün Azaltılması (Usability & Cognitive Load Reduction)

Sistem, uzun saatler boyunca ekran karşısında kod yazan ve araştırma yapan profesyonellerin kullanımına sunulacaktır. Bu nedenle arayüz ve kullanıcı deneyimi (UX), gereksiz görsel karmaşadan uzak, minimalist ve işlevsel bir yapıda tasarlanmalıdır. İçeriklerin okunabilirliğini maksimize etmek adına; koyu tema (dark mode) desteği, programlama dillerine göre otomatik sözdizimi vurgulama (syntax highlighting) yeteneği ve geliştiricilerin aşina olduğu klavye kısayollarını destekleyen zengin metin editörleri sistemin temel kullanılabilirlik hedeflerindendir.

2. Üst Düzey Yazılım Mimarisi

Bu bölümde, Yazılımcı Birleştirme Platformu (DevSözlük) sisteminin temelini oluşturan üst düzey mimari yapı, sistemin bağımsız alt bileşenlere (katmanlara) ayrıştırılma stratejisi, kullanılacak donanım ve yazılım teknolojilerinin eşleşimi, kalıcı verilerin nasıl yönetileceği, güvenlik politikaları ile sistemin sınır koşullarındaki (başlatma, sonlandırma, hata durumları) davranış biçimleri kapsamlı bir şekilde ele alınmaktadır. Mimarinin temel felsefesi; yüksek kohezyon (high cohesion) ve düşük bağımlılık (low coupling) prensipleri çerçevesinde, gelecekteki genişletmelere tamamen açık, sürdürülebilir ve esnek bir altyapı kurmaktır.

2.1. Alt Sistem Ayrıştırması



Şekil 2.1. DevSözlük Bileşen/Katman Diyagramı

Sistem, geliştirme süreçlerindeki karmaşıklığı yönetmek, farklı uzmanlık alanlarındaki geliştiricilerin (frontend, backend, veritabanı yöneticisi) birbirlerini engellemeden eşzamanlı çalışabilmesini sağlamak ve "Sorumlulukların Ayrılığı" (Separation of Concerns) prensibini uygulamak amacıyla N-Katmanlı İstemci-Sunucu (N-Tier Client-Server) mimarisi temel alınarak tasarlanmıştır. Bu yaklaşım doğrultusunda DevSözlük platformu, mantıksal ve fiziksel sınırlarla birbirinden izole edilmiş üç temel alt sisteme ayrıştırılmıştır:

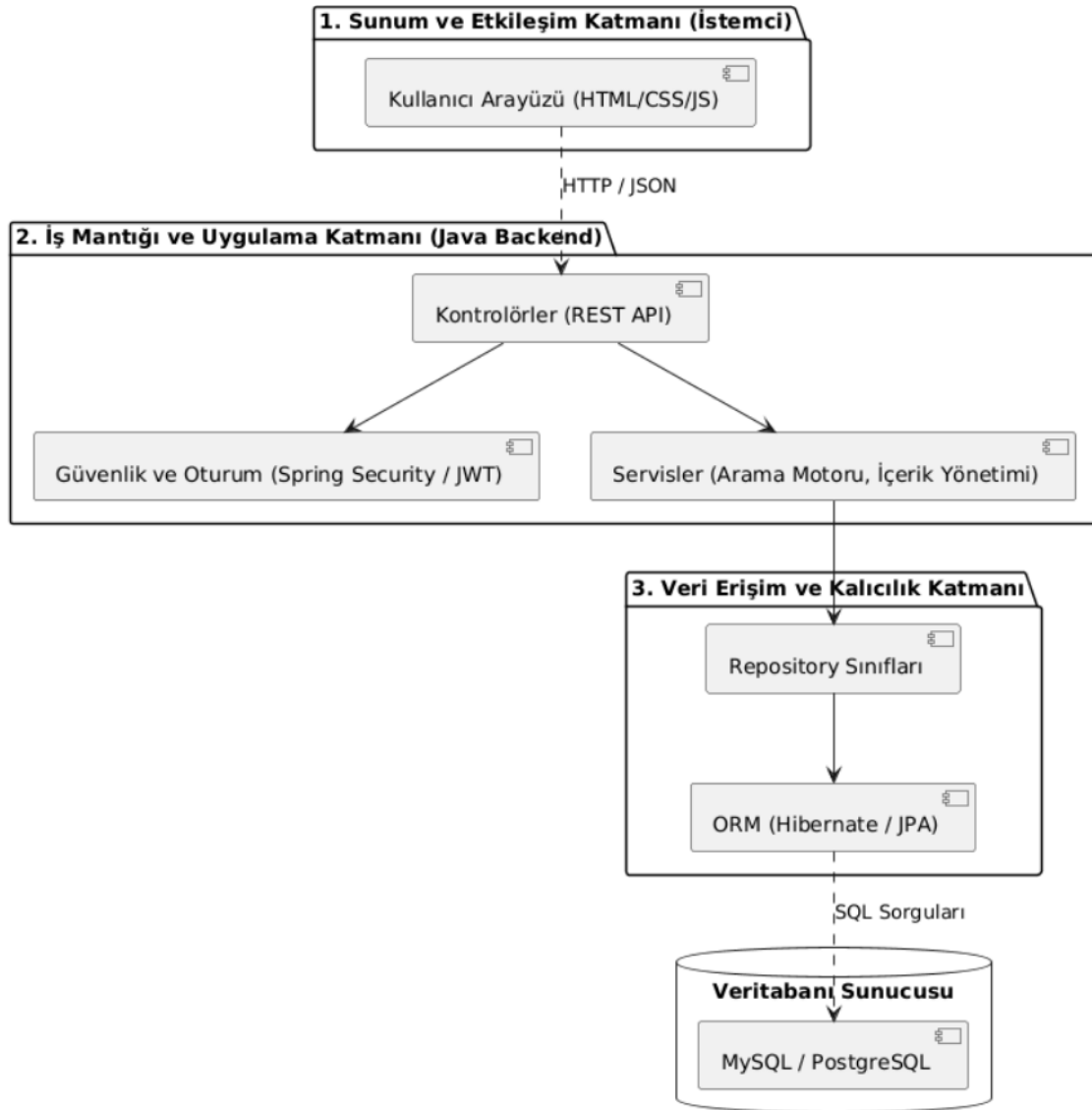
1. Sunum ve Etkileşim Katmanı (Presentation / Client Layer): Son kullanıcıların (Ziyaretçi, Standart Kullanıcı, Admin) sistemle doğrudan etkileşime girdiği en üst katmandır. Bu alt sistem, kullanıcı arayüzünün (UI) oluşturulmasından, istemci tarafındaki veri doğrulama (client-side validation) işlemlerinden ve sunucudan gelen JSON veya HTML formatındaki verilerin anlamlı bir şekilde ekrana yansıtılmasından

(render) sorumludur. Yazılımcıların kod parçacıklarını okunabilir bir şekilde görebilmesi için entegre edilecek sözdizimi vurgulayıcılar (syntax highlighters) ve arama çubuğu tetikleyicileri bu katmanın yönetimindedir. Bu katman, iş mantığını bilmez; yalnızca Uygulama Katmanı ile RESTful API'ler veya HTTP istekleri üzerinden asenkron olarak haberleşir.

2. İş Mantığı ve Uygulama Katmanı (Business Logic / Application Layer): Sistemin beynini oluşturan bu alt sistem, tüm çekirdek algoritmaların, güvenlik kontrollerinin ve kullanım durumlarının (Use Cases) işletildiği merkezdir. İstemciden gelen istekler (örneğin; UC03: Giriş Yap, UC06: Başlık Aç, UC08: Cevap Yaz) bu katmanda karşılanır. Gelen veriler sunucu tarafında sıkı bir doğrulamadan geçirilir. Kimlik doğrulama (Authentication), şifrelerin hashlenmesi (UC02), kullanıcı yetki kontrolleri (Authorization - UC16) ve arama motoru eşleştirme algoritmaları bu katmanın sorumluluğundadır. İlerleyen aşamalarda sisteme entegre edilecek olan yapay zeka tabanlı öneri sistemleri veya içerik filtreleme algoritmaları da bu katmanın bir parçası olarak çalışacaktır. Tasarımın bu şekilde modüler tutulması, arka planda kullanılan programlama dilinin (Java tabanlı arka yüz) versiyonu veya framework'ü (Örn: Spring Boot) güncellense bile diğer katmanların bu durumdan etkilenmemesini sağlar.

3. Veri Erişim ve Kalıcılık Katmanı (Data Access & Persistence Layer): İş Mantığı katmanında işlenen ve doğrulanmış olan verilerin (Kullanıcı hesapları, açılan başlıklar, yazılan entry'ler vb.) kalıcı olarak saklandığı, güncellendiği ve sorgulandığı alt sistemdir. Bu katman, sistemin ilişkisel bütünlüğünü (referential integrity) ve veri güvenliğini sağlar. İş mantığı katmanı, veritabanına doğrudan SQL sorguları göndermek yerine; ORM (Object-Relational Mapping) veya Repository tasarım desenleri aracılığıyla bu katmanla iletişim kurar. Bu sayede veritabanı yönetim sisteminin (RDBMS) ileride değiştirilmesi gerektiğinde (örneğin MySQL'den PostgreSQL'e geçiş) sistemin geri kalanında herhangi bir kod değişikliğine gerek kalmaz.

2.2. Donanım/Yazılım Eşlemesi



Şekil 2.2. DevSözlük Yerleşim (Deployment) Diyagramı

Sistemin, geliştirme ve canlı (production) ortamlarında sorunsuz çalışabilmesi için alt sistemlerin hangi donanım ve yazılım teknolojileri üzerinde konumlandırılacağı aşağıda detaylandırılmıştır. DevSözlük platformu, yüksek eşzamanlılık (concurrency) gerektiren bir bilgi paylaşım sistemi olduğundan, donanım ve yazılım bileşenleri performansı maksimize edecek şekilde eşleştirilmiştir.

- **İstemci Düğümü (Client Node / Frontend)**: Kullanıcıların sisteme eriştiği cihazları (bilgisayar, tablet veya mobil tarayıcılar) temsil eder. İstemci tarafında herhangi bir özel donanım kurulumuna gerek yoktur; modern bir web tarayıcısı (Chrome, Firefox, Safari vb.) yeterlidir. Kullanıcı arayüzü; HTML, CSS ve JavaScript teknolojileri kullanılarak tasarlanacak ve render edilecektir.

- **Uygulama Sunucusu (Application Server / Backend):** Sistemin iş mantığını, kimlik doğrulamalarını ve oturum yönetimini yürütecek olan ana merkezdir. Bu sunucu üzerinde **Java** tabanlı bir arka yüz (Backend) servisi koşacaktır. Java'nın güçlü multithreading (çoklu iş parçacığı) yapısı sayesinde, aynı anda binlerce kullanıcının başlık araması veya entry girmesi durumunda dahi sistem kararlılığı korunacaktır. Uygulama, Java Runtime Environment (JRE) destekleyen Linux veya Windows tabanlı bir sunucu donanımı üzerinde barındırılacaktır.
 - **Veritabanı Sunucusu (Database Server):** Uygulamanın kalıcı verilerinin (başlıklar, cevaplar, kullanıcı rolleri) tutulacağı ve yönetileceği donanım düğümüdür. Verilerin ilişkisel yapısı gereği bu sunucu üzerinde **MySQL veya PostgreSQL** gibi güçlü bir ilişkisel Veritabanı Yönetim Sistemi (RDBMS) çalıştırılacaktır. Veritabanı sunucusu, okuma hızını artırmak amacıyla SSD (Solid State Drive) tabanlı depolama birimleriyle desteklenmelidir.
-

2.3. Kalıcı Veri Yönetimi

Platformda üretilen içeriklerin (başlıklar ve entry'ler), kullanıcı hesaplarının ve sistemdeki yetki rollerinin kaybolmadan, güvenli ve tutarlı bir şekilde saklanması gerekmektedir. DevSözlük projesinde verilerin tutulması için **MySQL veya PostgreSQL** gibi ilişkisel bir veri tabanı (RDBMS) kullanılacaktır. Bu teknolojilerin seçilme gerekçesi; sistemdeki "Kullanıcı -> Başlık -> Cevap" şeklindeki katı hiyerarşik yapının (kompozisyon ve kalıtım ilişkilerinin) ilişkisel veritabanı tablolarıyla (Primary Key ve Foreign Key mantığıyla) en yüksek tutarlılıkta (ACID prensipleri) modellenebilmesidir.

Kalıcı veri yönetimi stratejisinin temel prensipleri şunlardır:

- **Veri Güvenliği ve Şifreleme (UC02):** Kullanıcıların sisteme kayıt olurken girdikleri düz metin şifreler, veritabanına doğrudan kaydedilmeyecektir. Veritabanı güvenliğini sağlamak amacıyla şifreler, Java arka yüzünde BCrypt gibi bir algoritma ile şifrelenerek (hashleme) veritabanına güvenli bir formatta yazılacaktır.
- **Parça-Bütün İlişkisi (Composition) ve Veri Bütünlüğü:** UML sınıf diyagramında belirtildiği üzere, bir "Başlık" nesnesi ile altındaki "Cevap" (Entry) nesneleri arasında güçlü bir kompozisyon ilişkisi vardır. Kalıcı veri yönetiminde bu kurala sıkı sıkıya bağlı kalınacaktır; yani veritabanında bir "Başlık" silindiğinde (Cascade Delete), o başlığa bağlı olan ve anlamsız kalacak tüm "Cevap" (Entry) kayıtları da otomatik olarak silinerek veri tabanında "yetim (orphan)" verilerin kalması engellenecektir.

- **Arama Optimizasyonu:** Kullanıcıların sistemdeki mevcut başlık isimlerine göre arama yapmasını sağlayan (UC09) Arama Motoru servisi, veritabanı üzerinde yüksek I/O işlemi oluşturacaktır. Bu yükü hafifletmek için, MySQL/PostgreSQL tarafında başlık isimlerinin tutulduğu kolonlara özel indekslemeler (B-Tree vb.) uygulanarak arama sorgularının veri tabanı seviyesinde hızlandırılması sağlanacaktır.

2.4. Erişim Kontrolü ve Güvenlik

DevSözlük platformu, teknik bilgi paylaşımının ötesinde, binlerce geliştiricinin entelektüel üretimini (kod parçacıkları, mimari çözümler) barındıran bir ekosistemdir. Bu bağlamda, sistemin güvenliği ve kullanıcı yetkilerinin yönetimi, mimarinin en kritik sacayaklarından birini oluşturmaktadır. Sistemdeki erişim kontrolü, "Rol Tabanlı Erişim Kontrolü" (Role-Based Access Control - RBAC) tasarım deseni temel alınarak Java arka yüzünde (backend) sıkı bir şekilde denetlenecektir.

Erişim kontrol mekanizması, istemci (client) tarafındaki arayüz gizlemelerine güvenmez; tüm yetki kontrolleri (Authorization - UC16) ve oturum denetimleri (Session Check - UC07) doğrudan Java sunucusu üzerinde, veritabanı ile eşzamanlı olarak gerçekleştirilir. Sistemde tanımlı olan temel aktörler ve bu aktörlerin yetki sınırları aşağıdaki erişim kontrol matrisinde özetlenmiştir:

Kullanım Durumu / İşlem	Ziyaretçi	Standart Kullanıcı	Admin
	✓	—	—
UC03 — Giriş Yap	✓	—	—
	—	✓	✓
UC09 — Başlık Ara	✓	✓	✓
	✓	✓	✓
UC06 — Başlık Aç	—	✓	✓
	—	✓	✓
UC13 — İstenmeyen İçeriği Sil	—	—	✓
	—	—	✓
UC15 — Entry Sil	—	—	✓
	—	—	✓

✓ = Erişim izni var — = Erişim izni yok (403 Forbidden)

- **Ziyaretçiler (Anonim Kullanıcılar):** Sisteme kimlik doğrulaması (login) yapmamış, "salt okunur" (read-only) yetkiye sahip kullanıcılardır. Arama motorunu kullanabilir (UC09), açılmış başlıkları ve altındaki cevapları sayfalandırılmış biçimde (UC12) görüntüleyebilirler. İçerik üretme veya oylama yetkileri (403 Forbidden) yoktur.
- **Standart Kullanıcılar:** Sisteme kayıt olmuş (UC01) ve Java sunucusu tarafından kimliği doğrulanarak (Authentication - UC04) aktif bir oturum (session) atanmış yazılımcılardır. Sistemin ana üretici gücüdür. Kendi adlarına başlık açabilir (UC06), mevcut başlıklara cevap (entry) yazabilir (UC08) ve topluluktaki diğer cevapları oylayabilirler.
- **Adminler (Yöneticiler / Moderatörler):** Platformun düzenini ve kalitesini korumakla görevli, en üst düzey (superuser) yetkilere sahip gruptur. Standart kullanıcıların tüm yetkilerine ek olarak; platform kurallarını ihlal eden, spam veya zararlı kod içeren başlıkları (UC14) ve tekil entry'leri (UC15) sistemden kalıcı veya mantıksal olarak (soft delete) silme yetkisine sahiptirler.

Güvenlik Politikaları ve Uygulama Mimarisi:

Güvenlik mimarisi sadece yetkilendirme ile sınırlı değildir. Aşağıdaki siber güvenlik politikaları sistemin çekirdeğine entegre edilmiştir:

1. **Parola Kriptografisi (UC02):** Hiçbir kullanıcının şifresi veritabanında düz metin (plain-text) olarak saklanmaz. Şifreler, Java tabanlı güvenlik kütüphaneleri (örn. BCrypt veya PBKDF2) kullanılarak rastgele "salt" (tuzlama) değerleriyle birlikte hashlenerek MySQL/PostgreSQL veritabanına yazılır.
2. **Girdi Sanitizasyonu ve Zafiyet Koruması:** Platform, kullanıcıların doğrudan kod bloğu paylaşmasına izin verdiği için XSS (Cross-Site Scripting) ve SQL Injection saldırılarına karşı yüksek risk altındadır. Bu riskleri önlemek amacıyla, istemciden gelen tüm veriler Java sunucusuna ulaştığı anda ORM (Object-Relational Mapping) araçları ve XSS filtreleme kütüphaneleri üzerinden geçirilerek sanitize edilir (temizlenir).

2.5. Küresel Kontrol Akışı

Sistemin "Küresel Kontrol Akışı" (Global Control Flow), kullanıcı etkileşimlerinden doğan olayların (events) istemciden sunucuya, oradan veritabanına ve tekrar kullanıcıya dönüşüne kadar izlediği yaşam döngüsünü ifade eder. DevSözlük sistemi, modern web uygulaması standartlarına uygun olarak **İstek-Yanıt (Request-Response) güdümlü, Senkron ve Yarı-Asenkron (AJAX tabanlı)** bir kontrol akışına sahiptir.

Sistem içindeki tipik bir içerik üretme sürecinin (Örneğin: Bir standart kullanıcının başlık açması - UC06) kontrol akışı şu adımları izler:

1. **Kullanıcı Girdisi ve İstemci Doğrulaması:** Kullanıcı arayüzde (HTML/JS) başlık adını girer ve "Gönder" butonuna tıklar. Tarayıcı, başlığın boş olup olmadığını veya karakter sınırlarını (client-side validation) kontrol eder. Başarılıysa, veriyi HTTP/HTTPS protokolü üzerinden JSON veya Form-Data formatında sunucuya iletir.
 2. **Sunucu Karşılması ve Oturum Kontrolü (Middleware/Interceptor Geçişi):** İstek, Java tabanlı uygulama sunucusuna ulaşır. İstek doğrudan iş mantığına iletilmeden önce bir "Filtre/Kesici (Interceptor)" katmanından geçer. Burada UC07 (Oturum Kontrolü) devreye girer. Sunucu, isteği yapan kullanıcının aktif ve geçerli bir oturumu olup olmadığını denetler. Oturum yoksa, işlem reddedilir ve kullanıcıya hata mesajı / giriş ekranı yönlendirmesi gönderilir.
 3. **İş Mantığının İşletilmesi (Controller ve Service Katmanı):** Oturum geçerliyse, veri Java'daki ilgili Controller (Yönlendirici) sınıfına aktarılır. Bu aşamada başlığın daha önce açılıp açılmadığı (mükerrer kayıt kontrolü) veya içerikte yasaklı kelime olup olmadığı gibi iş kuralları (business rules) işletilir.
 4. **Veri Erişim Katmanı ve Kalıcılık (Repository/DAO):** İş kurallarından başarıyla geçen veri, Data Access Object (DAO) veya Repository tasarım deseni kullanılarak ORM üzerinden güvenli SQL sorgularına (INSERT) dönüştürülür. MySQL/PostgreSQL veritabanı bu sorguyu işler, yeni kaydı oluşturur ve işlemin başarılı olduğuna dair sunucuya onay (acknowledgment) döner.
 5. **Arayüzün Güncellenmesi (Yanıt Döngüsü):** Java sunucusu, veritabanından aldığı onayı HTTP 200/201 (Başarılı/Oluşturuldu) durum koduyla birlikte HTML render ederek veya JSON nesnesi olarak istemciye geri gönderir. İstemci (tarayıcı), bu yanıtı alarak kullanıcıyı yeni açılan başlığın sayfasına yönlendirir (Redirect).
-

2.6. Sınır Koşulları (Boundary Conditions)

Sistemin "Mutlu Yol" (Happy Path) senaryoları dışında kalan olağandışı durumlarda, aşırı yüklenmelerde veya sistemin başlatılıp sonlandırılması süreçlerinde göstereceği tepkiler sınır koşulları olarak adlandırılır. DevSözlük platformunun veri kaybı yaşamadan ve güvenlik zafiyeti oluşturmadan bu sınır durumlarını nasıl yöneteceğine dair mimari politikalar aşağıda detaylandırılmıştır.

2.6.1. Başlatma (Initialization)

Sistemin (Java uygulama sunucusunun) başlatılması sıradan bir işlem değil, bir "hazırlık ve doğrulama" (warm-up & validation) sürecidir. Uygulama ayağa kalkarken şu adımlar izlenir:

- **Bağlantı Sınamaları:** Sunucu ilk olarak yapılandırma dosyalarını okur ve MySQL/PostgreSQL veritabanı ile bağlantı havuzu (Connection Pool - Örn: HikariCP) oluşturmaya çalışır. Eğer veritabanı sunucusuna erişilemezse, uygulama eksik bir şekilde ayağa kalkmaz; önyükleme (boot) sürecini durdurarak işletim sistemine "Kritik Hata (Fatal Error)" kodu döndürür ve operasyon ekibine log bırakır.
 - **Önbellek (Cache) Isıtma:** Veritabanı bağlantısı başarılıysa, arama motorunun daha hızlı çalışabilmesi için sistemde en çok aranan başlıklar ve sabit sistem ayarları belleğe (RAM/Redis) yüklenir. Bu işlem tamamlandığında sistem, istemcilerden gelen HTTP isteklerini kabul etmeye (Listening) başlar.
-

2.6.2. Sonlandırma (Termination)

Sistemin güncellenmesi, sunucu bakımı veya beklenmeyen bir yeniden başlatma durumunda, kullanıcıların o an yazmakta olduğu cevapların (entry) veya oyların kaybolmaması hayati önem taşır. Bu nedenle sistemde "Anında Kapatma" (Hard Kill) yerine "**Zarif Sonlandırma**" (Graceful Shutdown) mimarisi uygulanır:

- Kapatma sinyali (SIGTERM) alındığında, Java sunucusu ağ trafiğini dinlemeyi durdurur ve yeni HTTP isteklerini reddeder (Kullanıcılara 503 Service Unavailable döner).
 - Halihazırda işlemde olan (veritabanına yazılmakta olan) entry kayıtlarının veya kimlik doğrulama işlemlerinin (aktif thread'lerin) bitmesi için sisteme belirli bir tolerans süresi (örn: 30 saniye) tanınır.
 - Veritabanı bağlantı havuzu güvenli bir şekilde kapatılarak olası veri bozulmalarının (data corruption) önüne geçilir ve sistem tamamen kapanır. Oturumu açık olan kullanıcıların session bilgileri JWT ile yönetildiği veya veritabanında tutulduğu için yeniden başlatmada kullanıcılar çıkış yapmış (logout) sayılmaz.
-

2.6.3. Hata Durumları (Failures)

Sistem çalışırken oluşabilecek beklenmeyen arızalar (veritabanı bağlantısının kopması, aşırı trafik nedeniyle bellek yetersizliği, API hataları), güvenlik ve kullanıcı deneyimi gözetilerek yönetilir:

- **Küresel Hata Yakalayıcı (Global Exception Handling):** Java uygulamasının herhangi bir katmanında bir istisna (Exception) fırlatıldığında, bu hata merkezi bir denetleyici (ControllerAdvice) tarafından yakalanır.
 - **Güvenlik İzolasyonu:** Beklenmeyen sunucu hatalarında (500 Internal Server Error), kullanıcıya kesinlikle ham kod hatası, veritabanı sorgusu veya "Stack Trace" gösterilmez. Bu tür bilgilerin sızması, saldırganlara sistemin iç yapısı hakkında ipucu verir (Information Disclosure zafiyeti). Kullanıcıya sadece jenerik ve kibar bir hata sayfası (Örn: "Şu an sunucularımızda bir problem var, lütfen daha sonra tekrar deneyin") render edilir.
 - **Asenkron Loglama:** Kullanıcıdan gizlenen bu teknik hata detayları, arka planda loglama araçları (SLF4J, Logback) kullanılarak asenkron olarak dosyalara veya merkezi hata izleme sistemlerine yazılır. Böylece geliştirici ekip hatanın kök nedenini (root cause) analiz edebilir.
-

3. Alt Düzey Tasarım

Bu bölümde, üst düzey mimari kararların uygulama seviyesine (kod ve nesne) nasıl indirgeneceği detaylandırılmaktadır. Sistemin alt düzey tasarımı; nesneye yönelik programlama (OOP) prensipleri çerçevesinde kurgulanan sınıf hiyerarşilerini, katmanlar arası veri taşıma stratejilerini (DTO - Data Transfer Objects) ve yazılımın sürdürülebilirliğini güvence altına alan tasarım desenlerini (Design Patterns) içermektedir.

3.1. Nesne Tasarım Kararları

Gereksinim analizinden sistem tasarımına geçiş aşamasında, performans ve sürdürülebilirlik arasında denge kurabilmek adına aşağıdaki mimari ödünleşim (trade-off) kararları alınmıştır:

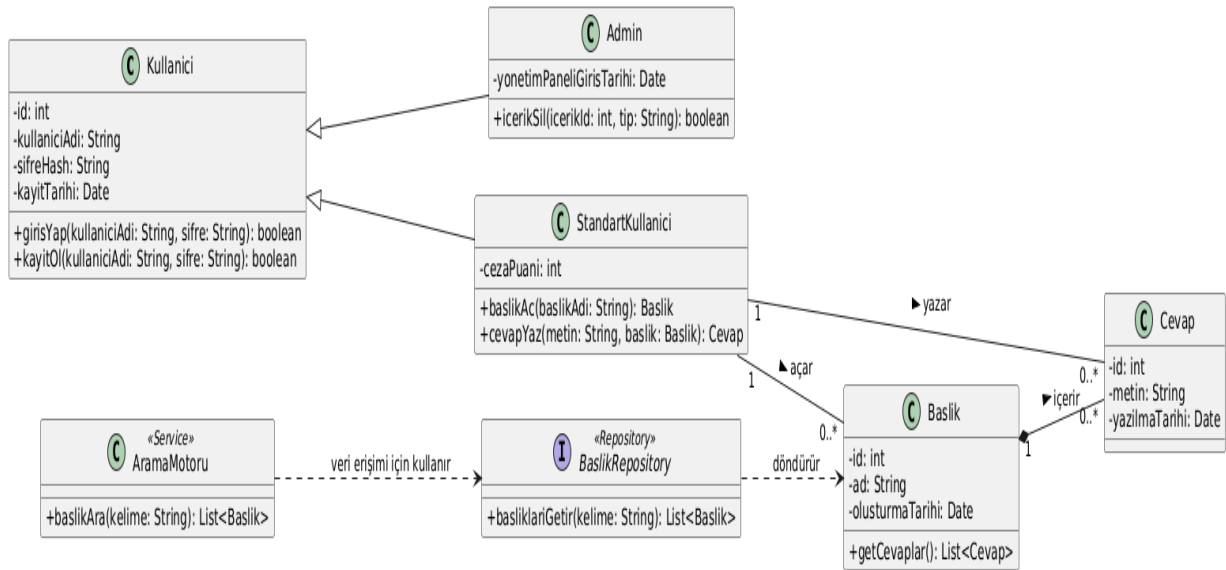
- **Veri Normalizasyonu vs. Performans (Denormalizasyon Stratejisi):** Bir "Başlık" (Baslik) nesnesinin altında binlerce "Cevap" (Entry) bulunabilir. Klasik veritabanı normalizasyon kurallarına göre, bir başlığın toplam entry sayısını bulmak için her seferinde veritabanına COUNT(*) sorgusu atılması gerekir. Ancak, DevSözlük gibi okuma yoğunluklu (read-heavy) platformlarda bu durum ciddi performans darboğazlarına (bottleneck) yol açar. Bu nedenle, performans hedeflerini önceleyen bir kararla kısmi **denormalizasyona** gidilmiş ve Baslik sınıfı içerisine

entrySayisi adında bir öznitelik (attribute) eklenmiştir. Yeni bir cevap girildiğinde (UC08), bu sayaç artırılarak okuma süreleri milisaniyeler seviyesine indirilmiştir.

- **Bellek Kullanımı vs. Hız (Lazy Loading Kararı):** İçerik okuma aşamasında, bir başlık çağrıldığında altındaki tüm cevapların (entry'lerin) tek seferde bellekten yüklenmesi (Eager Loading) sunucu hafızasını (RAM) hızla tüketir. Bu maliyetli işlem yerine, **Sayfalama (Pagination - UC12)** mantığı temel alınarak "Lazy Loading" stratejisi benimsenmiştir. İstemci her talepte yalnızca belirli sayıda (örn: 20 adet) entry verisini çekecektir.
- **İşlevsellik vs. Güvenlik Maliyeti:** Şifreleme işlemleri (BCrypt), sistemin işlemci (CPU) gücünü yoğun bir şekilde kullanan süreçlerdir. Geliştirme sürecine ek maliyet (cost) ve sunucuya işlem yükü getirmesine rağmen, hedef kitle olan yazılımcıların veri güvenliğini tehlikeye atmamak adına bu işlemci maliyeti göze alınmıştır (Security-First Approach).

3.2. Son Nesne Tasarımı

Sistemin alan modelini (Domain Model) oluşturan çekirdek varlık (Entity) sınıfları ve aralarındaki kalıtım/kompozisyon ilişkileri aşağıda detaylandırılmıştır. Bu varlıklar, doğrudan veritabanı tablolarına (ORM vasıtasıyla) eşlenecek nesnelerdir.



DevSözlük UML Sınıf Diyagramı

1. **Kullanici (Base Entity - Abstract Sınıf):** Sisteme dahil olan her bireyin ortak özelliklerini (ID, kullaniciAdi, sifreHash) barındıran temel sınıftır. Güvenlik prensipleri gereği doğrudan örneklenemez (instantiate edilemez), sadece kalıtım (inheritance) vermek için kullanılır.
2. **StandartKullanici (Kalıtım Alır: Kullanici):** Sistemin ana içerik üreticisidir. Başlık açma, cevap yazma ve oy kullanma yeteneklerine (metotlarına) sahiptir.

Veritabanında yazarın topluluktaki itibarını simgeleyen cezaPuani (veya karma puanı) gibi ekstra özniteliklere sahiptir.

3. **Admin (Kalıtım Alır: Kullanıcı):** Sistemin moderasyonundan sorumludur. Sadece bir Admin nesnesi, içeriği silmeye yarayan icerikSil(int id) fonksiyonunu çağırabilir.
4. **Baslik (Composition Root):** Platformdaki tartışma konularını temsil eder. Bir Başlık nesnesi, kendisini oluşturan yazarın bilgisine (Foreign Key) ve altındaki Cevap nesnelerinin listesine sahiptir.
5. **Cevap (Entry Component):** Başlıkların altında listelenen veri bloklarıdır. Oylama algoritmasının çalışabilmesi için oySayisi ve icerik olarak metin (ve opsiyonel olarak kod bloğu) özelliklerini taşır.

3.3. Paketler / Katmanlar

Uygulamanın Java ekosisteminde (örn: Spring Boot kullanılarak) geliştirileceği göz önüne alındığında, kod tabanını organize etmek için paketler belirli işlevlere göre sınıflandırılmıştır.

3.3.1. Dış Paketler (External Dependencies)

- **Java Persistence API (JPA / Hibernate):** Veritabanı (MySQL/PostgreSQL) ile Java nesneleri (Entities) arasındaki eşlemeyi (ORM) sağlamak için kullanılır.
- **Spring Security & JWT:** Sistemdeki yetkilendirme (Authorization) ve kimlik doğrulama (Authentication) işlemlerini yönetir. Güvenli oturum açma (UC03) için token üretiminden sorumludur.
- **BCrypt / Jasypt:** Kullanıcı şifrelerini (UC02) geri dönüştürülemez şekilde hash'lemek (özetlemek) için kullanılan dış kütüphane paketidir.

3.3.2. İç Paketler (Internal Packages)

Proje, "Feature-Based" (Özellik Odaklı) bir paketleme stratejisine sahiptir:

- com.devsozluk.auth: Ziyaretçi kaydı (UC01), şifreleme, doğrulama ve JWT token yönetimini barındıran servisleri içerir.
- com.devsozluk.topic: Yeni başlık açma (UC06), başlıkları listeleme, sayfalandırma ve Arama Motoru (UC09) işlevlerini (Repository ve Service katmanlarını) kapsar.
- com.devsozluk.entry: Başlıklar altına cevap (entry) yazma (UC08) ve oylama mekanizmasını barındırır.

- com.devsozluk.moderation: Sistem yöneticilerinin (Admin) işlem yaptığı, ihlalli içerikleri silme (UC13, UC14, UC15) yetkisini (UC16) çalıştıran yönetim paketidir.

3.4. Sınıf Arayüzleri

Sistemin iş mantığını yürütecek ve veri tabanı ile iletişim kuracak temel sınıfların (Entity, Service ve Repository) arayüzleri, nesne yönelimli programlama (OOP) ilkelerine uygun olarak tasarlanmıştır. Sınıflar arasındaki kalıtım (inheritance) ve bağımlılıklar, UML kuralları çerçevesinde kodlanacaktır. Her bir önemli sınıfın öznitelikleri (attributes) ve metot imzaları aşağıda detaylandırılmıştır.

1. Kullanici (Soyut Temel Sınıf / Base Entity) Sisteme giriş yapan herkesin ortak kimlik bilgilerini ve güvenlik verilerini taşıyan ana sınıftır. Güvenlik gereksinimleri doğrultusunda şifreler düz metin olarak barındırılmaz.

- **Öznitelikler:**

- - id: int: Veritabanındaki birincil anahtar.
- - kullanıcıAdi: String: Sisteme giriş için kullanılan benzersiz tanıtıcı.
- - sifreHash: String: Şifrenin tek yönlü şifrelenmiş (hash) hali.
- - kayıtTarihi: Date: Sisteme dahil olunan zaman damgası.

- **Metotlar:**

- + girisYap(kullanıcıAdi: String, sifre: String): boolean: Veritabanındaki hash ile kullanıcının girdiği şifreyi eşleştirir, başarılıysa oturum açar.
- + kayıtOl(kullanıcıAdi: String, sifre: String): boolean: Yeni bir kaydı hashleyerek sisteme ekler.

2. StandartKullanici (Kalıtım Alır: Kullanici) Platformun temel içerik üreticisidir. İçerik oluşturma ve etkileşim yetkilerine sahiptir.

- **Öznitelikler:**

- - cezaPuani: int: Kullanıcının sistemdeki kural ihlallerini tutar.

- **Metotlar:**

- + baslikAc(baslikAdi: String): Baslik: Sistemde yeni bir tartışma konusu nesnesi oluşturur ve veritabanına yazar.
- + cevapYaz(metin: String, baslik: Baslik): Cevap: İlgili başlığın altına yeni bir entry (Cevap) nesnesi bağlar.

3. Admin (Kalıtım Alır: Kullanici) Sistemin düzenini korumakla görevli, moderasyon yetkilerine sahip ayrıcalıklı sınıftır.

- **Öznitelikler:**

- - yönetimPaneliGirisTarihi: Date: Güvenlik denetimi için son yönetim paneli erişim kaydı.

- **Metotlar:**

- + icerikSil(icerikId: int, tip: String): boolean: "Baslik" veya "Cevap" tipindeki bir veriyi ve ona bağlı olan alt verileri sistemden kalıcı olarak temizler.

4. Baslik (Entity Sınıfı) Geliştiricilerin teknik sorunlarını tartışmaya açtıkları ana konteynerdir.

- **Öznitelikler:**

- - id: int, - ad: String, - olusturmaTarihi: Date.

- **Metotlar:**

- + getCevaplar(): List<Cevap>: Bir başlığa bağlı olan tüm cevapların listesini veritabanından döndürür.

5. Cevap / Entry (Entity Sınıfı) Başlıkların altına yazılan düz metin formatındaki teknik içeriklerdir.

- **Öznitelikler:**

- - id: int, - metin: String, - yazilmaTarihi: Date.

6. AramaMotoru (Service Sınıfı) ve BaslikRepository Sistemdeki içeriklerin aranmasını yöneten ve veritabanı işlemlerini soyutlayan arayüzlerdir.

- **AramaMotoru Metotları:**

- + baslikAra(kelime: String): List<Baslik>: İstemciden gelen kelime sorgusunu iş mantığından geçirerek veritabanı katmanına iletir.

- **BaslikRepository Metotları:**

- + basliklariGetir(kelime: String): List<Baslik>: ORM kullanarak veritabanına erişen ve sonuç döndüren Repository metodudur.

3.5. Tasarım Desenleri

DevSözlük platformunun arka yüz (backend) mimarisinde, kodun sürdürülebilirliğini, test edilebilirliğini ve genişletilebilirliğini artırmak amacıyla endüstri standartlarında kabul görmüş aşağıdaki tasarım desenleri (Design Patterns) uygulanmıştır:

1. **MVC (Model-View-Controller) Deseni:** Sistemin genel yapısı bu mimari kalıp üzerine inşa edilmiştir. Veritabanı tabloları (Baslik, Cevap) "Model"; iş mantığı ve yönlendirmeler "Controller"; kullanıcının tarayıcıda gördüğü HTML/JS arayüzü ise "View"ı temsil eder.
2. **Repository Deseni:** UML diyagramında yer alan BaslikRepository arayüzünde görüldüğü üzere, veritabanı (MySQL/PostgreSQL) ile olan CRUD (Oluştur, Oku, Güncelle, Sil) operasyonları doğrudan iş mantığına yazılmamış, Repository arayüzleri arkasında soyutlanmıştır. Bu sayede veritabanı teknolojisi değişse bile kodun geri kalanı etkilenmez.
3. **Service Deseni (Service Layer):** AramaMotoru gibi yardımcı sınıflar, kontrolör (Controller) ile Repository arasında köprü kuran servisler olarak tasarlanmıştır. Kompleks arama algoritmaları ve filtreleme kuralları (Business Logic) bu katmanda işletilir.
4. **Singleton Deseni:** Uygulama sunucusunun veritabanı bağlantı havuzu (Connection Pool) ve arama motoru servisinin sistem kaynaklarını tüketmemesi için, bu nesnelerin uygulama yaşam döngüsü boyunca sadece bir kez (tek bir instance olarak) üretilmesi (Singleton) sağlanacaktır.

Sözlük (Glossary)

- **Entry:** Kullanıcıların, bir konuyu açıklamak veya bir soruya çözüm üretmek amacıyla başlıkların altına girdikleri her bir metin veya kod bloklarıdır.
 - **Hashleme (Hashing):** Şifrelerin veya hassas verilerin, siber saldırılara karşı korunması amacıyla, tek yönlü matematiksel algoritmalar (örn: BCrypt) kullanılarak geri döndürülemez anlamsız metin dizilerine dönüştürülmesi işlemi.
 - **Moderasyon:** Platformun amacından sapmasını ve bilgi kirliliğini önlemek adına, yetkili yöneticilerin (Adminlerin) kurallara uymayan başlık veya cevapları sistemden temizlemesi işlemi.
 - **ORM (Object-Relational Mapping):** Java'daki nesne tabanlı sınıflar (Kullanıcı, Başlık vb.) ile MySQL/PostgreSQL gibi ilişkisel veritabanı tabloları arasında otomatik köprü kurarak doğrudan SQL yazma zorunluluğunu ortadan kaldıran teknoloji.
 - **Sayfalama (Pagination):** Bir başlığın altında çok fazla cevap olduğunda, arayüzün karmaşıklığından uzak kalması ve sistem performansının düşmemesi için içeriklerin sayfalara (örn: 20'şerli bloklar halinde) bölünerek gösterilmesi.
-

Referanslar

[1] ISO/IEC, “ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models,” 2011. [Online]. Available: <https://www.iso.org/standard/35733.html>

[2] Oracle, “Java Documentation,” Oracle Corporation. [Online]. Available: <https://docs.oracle.com/en/java/>

[3] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: <https://www.postgresql.org/docs/>

[4] Oracle, “MySQL Documentation,” Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/>

[5] VMware, “Spring Boot Reference Documentation.” [Online]. Available: <https://docs.spring.io/spring-boot/reference/>